

You are here >> LDraw.org [Documentation](#) > LDraw File Format > **LDraw File Format Specification**

Menu

You are here >> LDraw.org [Documentation](#) > LDraw File Format > **LDraw File Format Specification**



Documentation: LDraw File Format Specification

Table of Contents

- [Purpose](#)
- [LDraw Files](#)
 - [Text Encoding](#)
 - [File Name](#)
 - [Extension](#)
 - [Basic parsing / file content](#)
 - [Line Termination](#)
- [The LDraw Co-ordinate System](#)
- [The LDraw Unit \(LDU\)](#)
- [Line Types](#)
 - [Line Type 0](#)
 - [META Commands](#)
 - [Line Type 1](#)
 - [Line Type 2](#)
 - [Line Type 3](#)
 - [Line Type 4](#)
 - [Line Type 5](#)
- [Colours](#)
 - [Colour 16](#)
 - [Colour 24](#)
 - [Direct](#)

Maintained by: LDraw.org Standards Board

Revision: 1.0.2, 29-Apr-2012

This is an ratified, official LDraw.org document. Non-administrative changes can only be made with the approval of the maintainer.

Purpose

Describing LEGO® bricks and models in a virtual environment is a challenging prospect. In 1995 James Jessiman developed just such a system with his LDraw program and file format. Since then the LDraw file format has grown to be the standard by which most create LEGO® models on the computer.

The aim of this document is to compile and consolidate James's original file specification with selected additions made since.

LDraw Files

Text Based

All LDraw files are text based.

Text Encoding

LDraw files must be created in the UTF-8 encoding of the Unicode character set. The byte order mark (BOM) must not appear in LDraw files. LDraw software may choose to reject files containing the BOM.

Colours

- Blended Colours

- **META Commands**

- Header Statements
- STEP
- WRITE/PRINT
- CLEAR
- PAUSE
- SAVE
- File Type
- CATEGORY and KEYWORDS Statements
- MPD Statements
- BFC Statements
- Unofficial META Commands
- Adding New META Commands

Note:

Previous versions of the LDraw.org File Format did not specify a text encoding for LDraw files. Software not using UTF-8 has been widely deployed. Parsers are recommended to confirm that a file is valid UTF-8, and if not, to attempt either (a) opening the file using the Microsoft Windows code page most appropriate for the current language, or (b) open the file using Windows Code Page 1252. These fallback recommendations are encouraged for all software platforms, since they follow the behavior of the overwhelming majority of non-Unicode LDraw editors.

File Name

The requirement for LDraw files to conform to the DOS 8.3 filename convention has been removed. Filenames should not exceed 255 characters (including the extension). Special characters, such as &, #, |, and ?, should be avoided as they may also cause cross-platform issues and create problems when used in URLs. It is noted that long filenames and filenames with non-DOS permitted characters may not work with (older) software that does not fully support this specification.

Extension

All LDraw files carry the LDR (default), DAT or MPD extension.

Basic parsing / file content

An LDraw file consists of one command per line. There is no line length restriction. Each command consists of optional leading whitespace followed by whitespace-delimited tokens. Some commands also have trailing arbitrary data which may itself include internal whitespace; such data is not tokenized, but treated as single unit according to the command.

Whitespace is defined as one or more spaces (#32), tabs (#9), or combination thereof.

Lines may also be empty or consist only of whitespace. Such lines have no effect.

If a line is non-empty, the first token must be an integer from the list of valid Line Type numbers. This number dictates further parsing of tokens for that line. The parsing rules per linetype follow below.

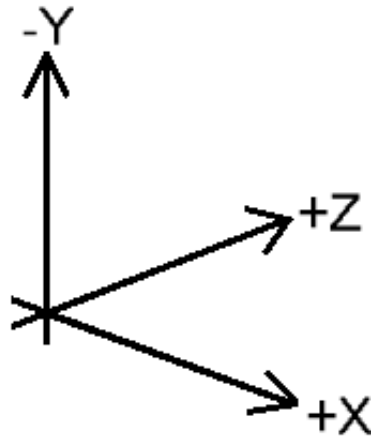
Line Termination

All lines in the file must use the standard DOS/Windows line termination of <CR><LF> (carriage return/line feed). The file is permitted (but not required) to end with a <CR><LF>. It is recommended that all LDraw-compliant programs also be capable of reading files with the standard Unix

line termination of <LF> (line feed).

The LDraw Co-ordinate System

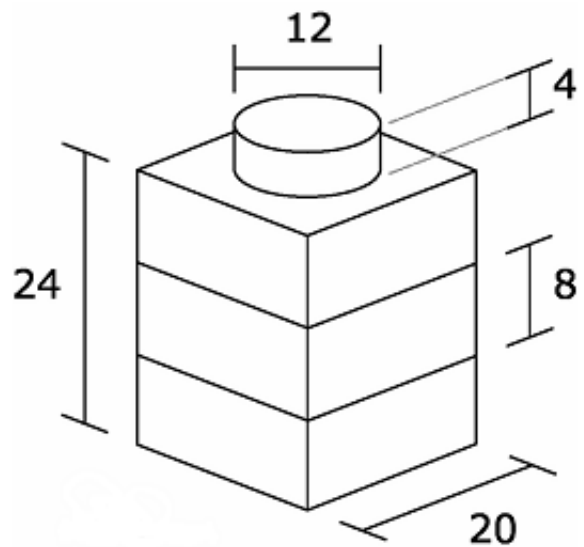
LDraw uses a right-handed co-ordinate system where -Y is "up".



The LDraw Unit (LDU)

LDraw parts are measured in LDraw Units (LDU)

- 1 brick width/depth = 20 LDU
- 1 brick height = 24 LDU
- 1 plate height = 8 LDU
- 1 stud diameter = 12 LDU
- 1 stud height = 4 LDU



Real World Approximations

- 1 LDU = 1/64 in
- 1 LDU = 0.4 mm

Note:

These real world approximations are just that: approximations. While they can be used for determining the LDraw equivalent of a portion of a physical part (for example, the curve of a surface), key points on such portions should be verified by reference to interactions with real parts. For example, it is usually possible to

build a framework of bricks and plates to "hug" the curve and act as reference points. When using these dimensions to determine a pattern they should only be used for small areas/detail. Large areas should be measured in multiples of plate/brick dimensions with any "left-over" being converted with the above approximations.

Line Types

The line type of a line is the first number on the line. The line types are:

- 0: Comment or META Command
- 1: Sub-file reference
- 2: Line
- 3: Triangle
- 4: Quadrilateral
- 5: Optional Line

If the line type of the command is invalid, the line is ignored.

Line Type 0

Line type 0 has two uses. One use is a comment the other is as a META command.

If the first line of a file is a line type 0 the remainder of the line is considered the file title (see [Library Header specification](#)). This overrides any META commands on that line.

A comment line is formatted:

```
0 // <comment>
```

or

```
0 <comment>
```

Where:

<comment> is any string

The form `0 // <comment>` is preferred as the `//` marker clearly indicates that the line is a comment, thereby permitting parsers to stop processing the line.

The form `0 <comment>` is deprecated.

META Commands

A META command is a statement used to tell an LDraw compatible program to do something. There are currently many official META commands and even more [unofficial META commands](#). In a META command, a keyword follows the line type in the line. The keyword must be in all caps. The generic META line format is:

0 !<META command> <additional parameters>

Where:

- ! is used to positively identify this as a META command. (Note: A few official meta commands do not start with a ! in order to preserve backwards compatibility, however, all new official META commands must start with a ! and it is strongly recommended that new unofficial meta-commands also start with a !)
- <META command> is any string in all caps
- <additional parameters> is any string. Note that if a META command does not require any additional parameter, none should be given.

Line Type 1

Line type 1 is a sub-file reference. The generic format is:

1 <colour> x y z a b c d e f g h i <file>

Where:

- <colour> is a number representing the colour of the part. See the [Colours section](#) for allowable colour numbers.
- x y z is the x y z coordinate of the part
- a b c d e f g h i is a top left 3x3 matrix of a standard 4x4 homogeneous transformation matrix. This represents the rotation and scaling of the part. The entire 4x4 3D transformation matrix would then take either of the following forms:

$$\begin{array}{cccc} / & a & d & g & 0 & \backslash \\ | & b & e & h & 0 & | \\ | & c & f & i & 0 & | \\ \backslash & x & y & z & 1 & / \end{array} \quad \begin{array}{cccc} / & a & b & c & x & \backslash \\ | & d & e & f & y & | \\ | & g & h & i & z & | \\ \backslash & 0 & 0 & 0 & 1 & / \end{array}$$

The above two forms are essentially equivalent, but note the location of the transformation portion (x, y, z) relative to the other terms.

- Formally, the transformed point (u', v', w') can be calculated from point (u, v, w) as follows:

$$\begin{aligned} u' &= a*u + b*v + c*w + x \\ v' &= d*u + e*v + f*w + y \\ w' &= g*u + h*v + i*w + z \end{aligned}$$

- <file> is the filename of the sub-file referenced and must be a [valid LDraw filename](#). Any leading and/or trailing whitespace must be ignored. Normal token separation is otherwise disabled for the filename value.

Line type 1 should never use the colour 24.

The complement colour is neither unique nor symmetric. The complement of the complement of colour X is not necessarily colour X. Using colour 24 for line type 1 will therefore lead to unexpected/undefined results.

Sub-files can be located in the LDRAW\PARTS sub-directory, the LDRAW\P sub-directory, the LDRAW\MODELS sub-directory, the current file's directory, a path relative to one of these directories, or a full path may be specified. Sub-parts are typically stored in the LDRAW\PARTS\S sub-directory and so are referenced as s\subpart.dat, while hi-res primitives are stored in the LDRAW\P\48 sub-directory and so referenced as 48\hires.dat

While there is no specified limit on how deep sub-files may be nested, there are probably practical limitations imposed by individual software programs.

There are many on-line references about transformation matrices, one such reference is [3D Transformations – Part 1 Matrices](#)

Line Type 2

Line type 2 is a line drawn between two points. The generic format is:

2 <colour> x1 y1 z1 x2 y2 z2

Where:

- <colour> is a number representing the colour of the part, typically this is 24 - the edge colour. See the [Colours section](#) for allowable colour numbers.
- x1 y1 z1 is the coordinate of the first point
- x2 y2 z2 is the coordinate of the second point

Line type 2 (and also 5) is typically used to edge parts. When used in this manner colour 24 must be used for the line. It should be remembered that not all renderers display line types 2 and 5

Line Type 3

Line type 3 is a filled triangle drawn between three points. The generic format is:

3 <colour> x1 y1 z1 x2 y2 z2 x3 y3 z3

Where:

- <colour> is a number representing the colour of the part. See the [Colours section](#) for allowable colour numbers.
- x1 y1 z1 is the coordinate of the first point
- x2 y2 z2 is the coordinate of the second point

- $x_3\ y_3\ z_3$ is the coordinate of the third point

See also the comments about polygons at the end of the Line Type 4 section.

Line Type 4

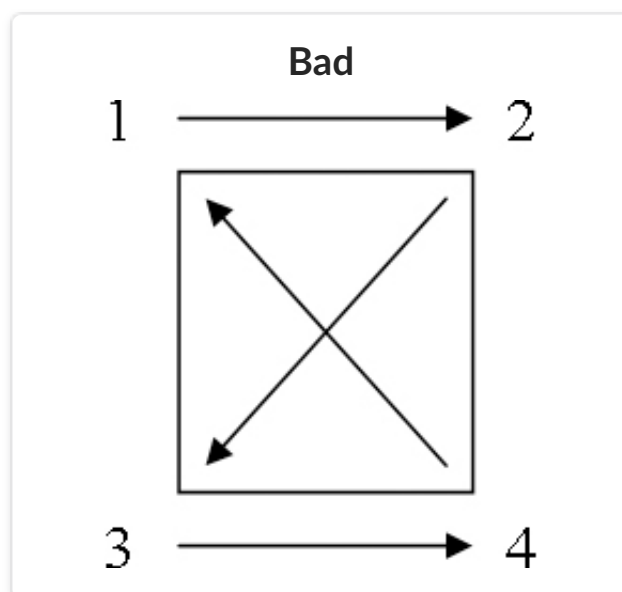
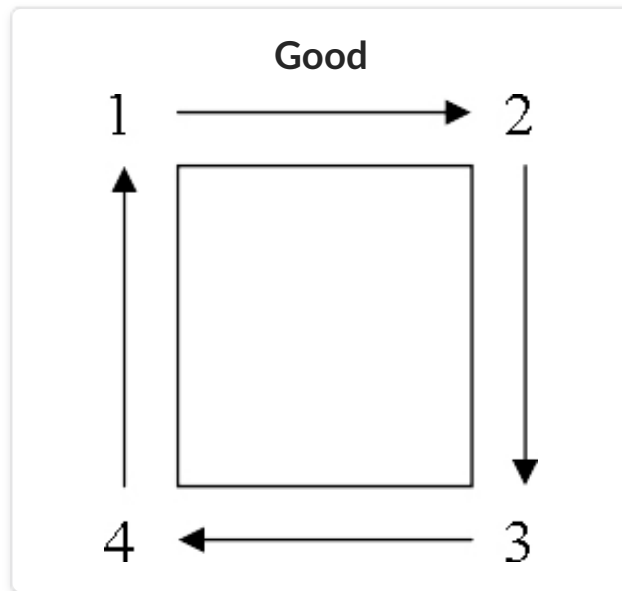
Line type 4 is a filled quadrilateral (also known as a "quad") drawn between four points. The generic format is:

4 <colour> $x_1\ y_1\ z_1\ x_2\ y_2\ z_2\ x_3\ y_3\ z_3\ x_4\ y_4\ z_4$

Where:

- <colour> is a number representing the colour of the part. See the [Colours section](#) for allowable colour numbers.
- $x_1\ y_1\ z_1$ is the coordinate of the first point
- $x_2\ y_2\ z_2$ is the coordinate of the second point
- $x_3\ y_3\ z_3$ is the coordinate of the third point
- $x_4\ y_4\ z_4$ is the coordinate of the fourth point

While no specific order (winding) is required, the points of the quadrilateral must be declared in either a clockwise (CW) or counter-clockwise (CCW) order (see [BFC statements](#)). The "good" example exhibits clockwise winding. If the points were given in the order 1, 4, 3, 2, they would exhibit counter-clockwise winding and also be considered as "good". The "bad" example illustrates a "bow-tie" or "hour-glass" quad.



The four points must lie within a plane. Quads should not contain three co-linear points (they should be reduced to a triangle). Quads must not be concave (two or more other polygons must be used instead).

Co-planar polygons (line types 3 and 4) may overlap only if they are the same colour. Any such overlap must be kept to a minimum as it affects the render quality of transparent parts.

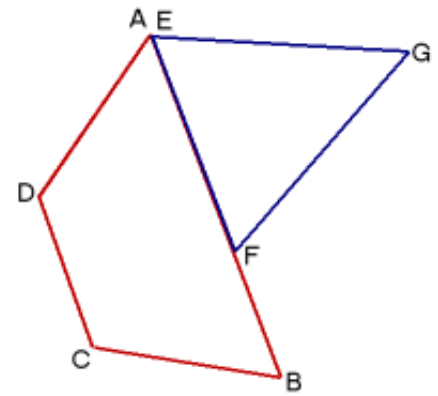
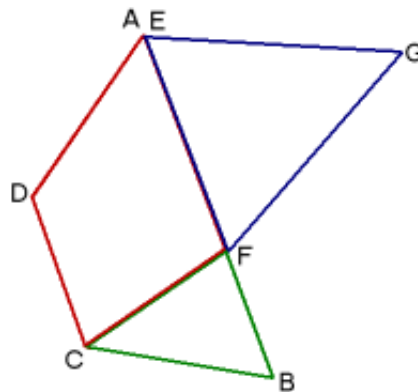
Polygons may intersect, but any such intersection must be kept to a minimum as it affects the render quality of transparent parts.

Where possible, abutting polygons should share common vertices. That is, if polygons ABCD and EFG abut along the line AEFB, A and E should be the same point and F and B should be the same point.

If A and E are the same point but F and B are not the same point, the pixel approximations of lines A'B' and A'F' (where A' represents the pixel point of A - which will be subject to integer truncation) may not be the same along the length A'F'. This will result in holes being visible in the part, especially at high magnification/resolution, and is the primary cause of "dot creep" when rotating parts.

Where practical, if A and E (and/or F and B) are not the same point, one or other or both of the polygons should be decomposed into two or more smaller polygons such that they share common vertices.

In the



diagram, ABCD has been decomposed into the quad AFCD and the tri FBC

It is strongly recommended that polygons do not use colour 24 (the complement colour).

(The rendition of a colour 24 polygon in a main file is implementation dependant (and may not be rendered at all), while the rendition of a colour 24 polygon in a sub-file is configurable, and so may not give the expected results in all situations/renderers.)

Line Type 5

Line type 5 is an optional line. The generic format is:

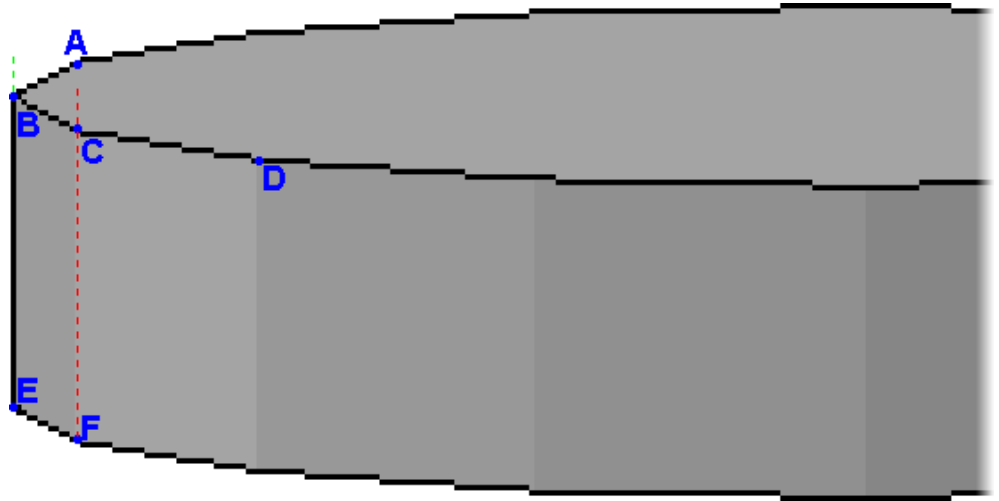
5 <colour> x1 y1 z1 x2 y2 z2 x3 y3 z3 x4 y4 z4

Where:

- <colour> is a number representing the colour of the part, typically this is 24 - the edge colour. See the [Colours section](#) for allowable colour numbers.
- x1 y1 z1 is the coordinate of the first point
- x2 y2 z2 is the coordinate of the second point
- x3 y3 z3 is the coordinate of the first control point
- x4 y4 z4 is the coordinate of the second control point

With an optional line, a line between the first two points will only be drawn if the projections of the last two points (the control points) onto the screen are on the same side of an imaginary line through the projections of the first two points onto the screen.

The optional line can be a difficult concept to grasp. The best way to demonstrate how they work is through a picture.



5 24 Bx By Bz Ex Ey Ez Ax Ay Az Cx Cy Cz

A and C are on the same side of the green line through BE, so BE is drawn.

5 24 Cx Cy Cz Fx Fy Fz Bx By Bz Dx Dy Dz

B and D are *not on* the same side of the red line through CF, so CF is *not* drawn.

This serves to "outline" the edges of the curved surface, which is the intent of optional lines.

As seen above, the control points usually can be chosen from already known points in the object. Since they are never drawn, they can be located anywhere, as long as they have the right controlling properties.

Optional lines have the same colour rules as normal lines (Line Type 2)

Colours

The official colours are defined via Colour Definition statements in the Idconfig.ldr file

See the [LDraw.org Colour Definition Language Extension](https://ldraw.org/Colour-Definition-Language-Extension) for a description of the Colour Definition Meta statement

The colour numbers 16 and 24 are reserved and must not be used in Colour Definition statements.

Colour 16

Colour 16 is referred to as the "main colour" or "current colour". When a sub-file is referenced by another file all the colour 16 command lines are displayed using the colour of the line that referenced it.

Example: 1 4 0 0 0 1 0 0 0 1 0 0 0 1 part.dat

All the colour 16 command lines in part.dat will be displayed using colour 4

Colour 24

Colour 24 is referred to as the "complement colour" or "edge colour". Colour 24 is most commonly used for line types 2 (line) and 5 (optional line). When a sub-file is referenced by another file all the colour 24 command lines are displayed using the complement colour of the line that referenced it. The complement colour of a given colour is defined in the Colour Definition for that colour.

Example: 1 4 0 0 0 1 0 0 0 1 0 0 0 1 part.dat

All the colour 24 command lines in part.dat will be displayed using the complement colour of colour 4

Use of colour 24 for line types other than 2 and 5 is strongly discouraged.

See the [LDraw Colour Chart](#) for a complete list of the LDraw colour number equivalents to the official LEGO colours.

Direct Colours

Direct Colours are colour numbers that represent RGB values using the form 0x2RRGGBB, where RRGGBB is the exact RGB value in hexadecimal notation. For example, 0x2008000 would produce dark green with RGB of (0 red, 128 green, 0 blue). All letters (A-F) used in the specification of RGB values must be in upper case.

LDraw compatible visualization applications shall be capable of displaying Direct RGB Values from Patterned Parts that contain these colours. For applications that support part authoring, it is recommended that the program provide a graphical aid for choosing a Direct RGB Value. The use of Direct RGB Values is discouraged in model authoring. Therefore, applications that support model authoring should favour colours from LDConfig.ldr.

Blended Colours

LDraw compatible visualization applications that once supported dithered colours should continue to do so in the display of parts and models that may contain a dithered colour code. However, dithered colours have been made obsolete and will not be contained within any new official parts. These colours should be removed from any graphical aids for choosing colours, and should not be added to any new graphical aids for choosing colours. New applications are not required to support these colours.

These "dithered" colours are defined based on a modification of ldraw.exe's original behavior that uses RGB blending instead of dithering. The original ldraw.exe drew them as a 50/50 checkerboard dither of two of its standard 16 colours. Since modern computers all support true colour, they will instead be drawn as a blend of the two input colors, which is calculated as the average of two original ldraw.exe colours.

The two input colour indexes are calculated as follows:

$$n_1 = (n_b - 256) / 16$$

$$n_2 = (n_b - 256) \bmod 16$$

Where:

- n_1 is the index of input colour 1
- n_b is the index of the blended color
- n_2 is the index of input colour 2

Once n_1 and n_2 are determined, the colour of the blended colour is calculated using the following equations:

$$R_b = (R_1 + R_2) / 2$$

$$G_b = (G_1 + G_2) / 2$$

$$B_b = (B_1 + B_2) / 2$$

Where:

- R_b , G_b , B_b are the red, green, and blue components of the blended colour
- R_1 , G_1 , B_1 are the red, green, and blue components of the first input colour
- R_2 , G_2 , B_2 are the red, green, and blue components of the second input colour.

The red, green, and blue components of the original 16 ldraw.exe colours are as follows:

0: 51, 51, 51

1: 0, 51, 178

2: 0, 127, 51

3: 0, 181, 166

4: 204, 0, 0

5: 255, 51, 153

6: 102, 51, 0

7: 153, 153, 153

8: 102, 102, 88

9: 0, 128, 255

10: 51, 255, 102

11: 171, 253, 249

12: 255, 0, 0

13: 255, 176, 204

14: 255, 229, 0

15: 255, 255, 255

Going the other direction, the blended colour's colour code is calculated using the following equation:

$$n_b = n_1 * 16 + n_2 + 256$$

Where:

- n_b is the index of the blended colour
- n_1 is the index of the first input colour
- n_2 is the index of the second input colour

META Commands

Meta commands (or meta statements) are a powerful way to extend the File Format Specification to enable the inclusion of meta data (such as author name, keywords, category, etc) and processing instructions (such as step, bfc, etc) within the data file. Reference the [formal META command definition](#) in the Line Type 0 section

For historical reasons, the original meta commands are defined as part of the file format specification, while others are defined in seperate language extension documents.

Header Statements

See the [Official Library Header Specification](#)

STEP

Marks the end of a building step

Format:

0 STEP

The Step command causes the original LDraw program to pause until the Enter key is pressed. Additionally a bitmap of the currently displayed parts in the model is saved to the LDRAW\BITMAP directory. Therefore, for the original LDraw program, 0 STEP is shorthand for 0 SAVE followed by 0 PAUSE

Other programs may interpret this META command differently.

WRITE/PRINT

Displays a message

Format:

0 WRITE <message>

Or

0 PRINT <message>

Where:

<message> is a string of any length.

The Write or Print commands cause the original LDraw program to display a message to the screen. This message is not saved on any bitmaps.

Other programs may interpret this META command differently.

CLEAR

Clears the screen

Format:

0 CLEAR

This command causes all previous drawn sub-files and line types to be cleared (erased). Any following line types are drawn as normal.

PAUSE

Pauses the drawing of a file

Format:

0 PAUSE

This command stops the further parsing of lines until the Enter key is pressed. Unlike the Step command, PAUSE does not save a bitmap of the currently displayed model to the LDRAW\BITMAP directory.

SAVE

Saves a bitmap of the current drawing

Format:

0 SAVE

Saves a bitmap of the current drawing in the LDRAW\BITMAP directory. Unlike the Step command, this command does not cause the program to pause.

File Type

Indicates the type of file (model, part, etc), and if it is part of the LDraw.org Parts Library.

Format:

0 !LDRAW_ORG <type> (qualifier(s)) (update-tag)

or

0 LDRAW_ORG <type> update-tag

or

0 Official LCAD <type> update-tag

or

0 Unofficial <type>

or
0 Un-official <type>

The first format (!LDRAW_ORG) is the [current standard for files in the official parts library](#). Older files still carry the second and third (Official LCAD) formats. The valid values for the <type> field are:

- All valid values specified in the [Official Library Header Specification](#) or the [Official Model Repository \(OMR\) Specification](#)
- File -
- Submodel -
- Element +
- Sub-part +
- Hi-Res Primitive +
- Alias +
- Cross-reference +

- = only used in unofficial files

+ = obsolete/deprecated, still appears on some files.

Some older official files do not include a type code on their File Type line. Unofficial files may have a variety of values after the initial tag. The exact format of the File Type line has changed over time. In general, parsers should consider this line to be case-insensitive and free-format.

CATEGORY and KEYWORDS Statements

See the [CATEGORY and KEYWORDS Language Extension](#)

MPD Statements

See the [MPD Language Extension](#)

BFC Statements

See the [LDraw Language Extension for Back Face Culling \(BFC\)](#)

Unofficial META Commands

See the [list of known META commands and the programs that use them](#)

Adding New META Commands

The following guidelines should be followed by developers when adding new META command functionality to their LDraw.org compatible programs:

- All new META commands should be based on the program name where this META command is first defined/used. For example, if your program is called LDParse and you want to add a META command called EXAMPLE, your command should appear in DAT

code as0 !LDPARSER EXAMPLE

- Avoid defining META commands which are similar in name but not equal in function to already existing META command. Doing this will avoid user confusion and possible erroneous output.

Table of Contents

- Purpose
- LDraw Files
 - Text Encoding
 - File Name
 - Extension
 - Basic parsing / file content
 - Line Termination
- The LDraw Co-ordinate System
- The LDraw Unit (LDU)
- Line Types
 - Line Type 0
 - META Commands
 - Line Type 1
 - Line Type 2
 - Line Type 3
 - Line Type 4
 - Line Type 5
- Colours
 - Colour 16
 - Colour 24
 - Direct Colours
 - Blended Colours
- META Commands
 - Header Statements
 - STEP